

Guía de Análisis Automatizado de Registros (Logs)

NetGuard Pro · Herramienta logsum

Resumen de errores y advertencias, conteo de problemas recurrentes y detección de patrones

Control del documento

Campo	Valor
Documento	Guía de análisis automatizado de logs de NetGuard Pro
Herramienta	logsum (Python, un solo archivo, sin dependencias externas)
Versión del documento	1.0
Fecha	5 de octubre de 2026
Responsable	[Nombre del responsable]
Revisado por	[Nombre del revisor]
Estado	Borrador [Aprobado]
Clasificación	[Interno / Confidencial]

Contenido

1. Introducción	3
1.1 Propósito de la guía	3
1.2 Alcance	3
1.3 Audiencia y requisitos previos	3
1.4 Resultados del análisis de referencia	3
2. Descripción de la herramienta	3
2.1 Descripción general	3
2.2 Características principales	4
2.3 Flujo de procesamiento	4
2.4 Formatos de log soportados	4
3. Guía paso a paso	4
Paso 1. Definir los objetivos	4
Paso 2. Recopilación de datos	5
Paso 3. Procesamiento previo de datos	5
Paso 4. Entrenamiento y refinamiento de la herramienta	6
Paso 5. Pruebas y evaluación	6
4. Principales desafíos y soluciones	7
5. Casos de uso potenciales	7
6. Instrucciones para el usuario	8
6.1 Instalación	8
6.2 Comandos básicos	8
6.3 Referencia de opciones	8
6.4 Cómo leer el informe	8
6.5 Solución de problemas	9
7. Escenarios de ejemplo	9
Escenario 1. Triage de un servidor inestable	9
Escenario 2. Revisión consolidada para detectar problemas sistémicos	9
Anexo A. Plantilla de registro de hallazgos	10
Anexo B. Lista de verificación antes de entregar un análisis	10
Anexo C. Glosario	11
Anexo D. Control de cambios	11

1. Introducción

1.1 Propósito de la guía

Esta guía explica cómo usar y mantener **logsum**, la herramienta que automatiza y agiliza el análisis de registros (logs) de NetGuard Pro. Su objetivo es resumir los datos de los logs, listar errores y advertencias únicos, contar los problemas recurrentes para resaltar los más comunes e identificar patrones o tendencias que delaten problemas sistémicos de mayor alcance.

Funciona como **plantilla reutilizable**: los campos entre [corchetes] se completan con los datos de su entorno, y los ejemplos usan el análisis real de seis archivos de log (servidor, actividad de usuario, depuración de aplicación y monitoreo de sistema).

1.2 Alcance

Incluye	No incluye
Análisis de archivos de log en texto, JSON y acceso web (también .gz y carpetas). Resumen por nivel, por archivo y por componente. Detección de patrones recurrentes, picos y tendencias. Informes en consola, Markdown y JSON. Interpretación opcional con IA (Claude).	Monitoreo en tiempo real ni alertas automáticas. Recolección centralizada de logs (agentes, SIEM). Corrección automática de incidentes. Análisis forense con valor legal o cadena de custodia.

1.3 Audiencia y requisitos previos

Dirigida a equipos de operaciones, desarrollo, soporte y seguridad. Se requiere Python 3.8 o superior, acceso de lectura a los logs y, solo para la función opcional de IA, el paquete `anthropic` y una clave de API.

1.4 Resultados del análisis de referencia

6 archivos analizados	180 entradas de log	32 errores	33 advertencias
---------------------------------	-------------------------------	----------------------	---------------------------

- **Server_Log_1** (25 abr) es el más crítico: 20 de sus 35 entradas son problemas, en menos de 3 horas.
- **Reincidentes**: DatabaseConnect (4 errores), ServiceCrash (4) y HighMemoryUsage (4 advertencias).
- **Temas sistémicos**: base de datos (unos 10 de 32 errores), red, saturación de memoria, CPU y disco, y servicios que caen.

2. Descripción de la herramienta

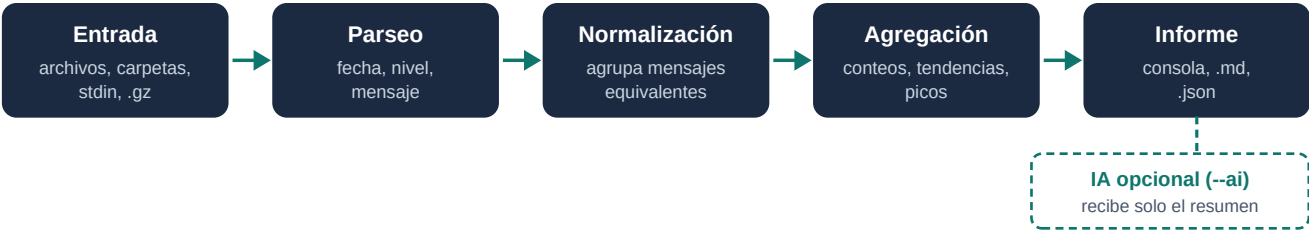
2.1 Descripción general

logsum es una utilidad de línea de comandos que lee los logs en *streaming*, agrupa los mensajes equivalentes y genera un informe legible. El análisis es **determinista** (basado en reglas): es rápido, repetible y se ejecuta localmente. La capa de IA es opcional y solo recibe el resumen, no los logs completos.

2.2 Características principales

Característica	Descripción
Resumen por nivel y archivo	Cuenta ERROR, WARNING, INFO y DEBUG, y desglosa los problemas por fuente.
Mensajes únicos agrupados	Normaliza IP, rutas, números, UUID y hexadecimales para agrupar mensajes que solo difieren en datos variables; indica cuántas variantes hubo.
Conteo de recurrentes	Ranking por frecuencia y porcentaje del total de problemas, más un resumen por componente (ej. DatabaseConnect).
Patrones y tendencias	Etiqueta cada patrón como NUEVO, RÁFAGA, CRECIENTE, DECRECIENTE o CONSTANTE; detecta picos anómalos y dibuja una línea de tiempo.
Gran volumen	Lectura en streaming y límite de 5,000 patrones en memoria. Medido: 1 millón de líneas (55 MB) en unos 23 s.
Formatos diversos	Texto con nivel, JSON lines, Apache/Nginx, syslog, trazas multilínea, archivos .gz, carpetas y entrada estándar.
Salidas	Consola, Markdown (--out) y JSON (--json) para integrarlo con otras herramientas.
Interpretación con IA (opcional)	--ai envía el resumen a Claude para obtener causas probables y acciones priorizadas.

2.3 Flujo de procesamiento



2.4 Formatos de log soportados

Formato	Ejemplo	Detección del nivel
Texto con nivel	2024-06-01 10:04:00 ERROR NullPointerException - ...	Explícito. WARN y WARNING se unifican.
JSON lines	{"time":"...", "level":"error", "msg":"..."}	Claves level, severity o lvl.
Acceso web	"GET /api/item/3 HTTP/1.1" 500	5xx = ERROR; 4xx = WARNING.
Syslog sin nivel	Jan 5 10:00:00 host sshd: Failed password	Inferido por palabras clave.
Trazas multilínea	Traceback ... / at com.empresa.Clase	Se adjuntan a la entrada anterior.

3. Guía paso a paso

El proceso sigue cinco pasos y es iterativo: los resultados de las pruebas (paso 5) alimentan nuevos ajustes (paso 4).

Paso 1. Definir los objetivos

Antes de ejecutar la herramienta, defina qué decisión debe apoyar el análisis y cómo sabrá que funcionó.

Objetivo	Indicador de éxito	Dónde se ve en el informe
Identificar los problemas más frecuentes	Top 10 de patrones con conteo y porcentaje	Secciones Errores y Advertencias
Detectar fallos sistémicos	Componentes que reinciden entre archivos o periodos	Componentes con más problemas
Detectar deterioro progresivo	Patrones CRECIENTE o CONSTANTE y picos	Patrones y tendencias; Picos
Reducir el tiempo de diagnóstico	De [X] horas a [Y] minutos (medición interna)	[A completar por el equipo]

Objetivo	Indicador de éxito	Dónde se ve en el informe
Mejorar la calidad del logging	Lista de ruido y brechas detectadas	Hallazgos (Anexo A)

Ejemplo NetGuard Pro: el objetivo fue identificar errores recurrentes y problemas sistémicos en seis archivos de log de distinto tipo.

Paso 2. Recopilación de datos

- Identifique las fuentes: logs de servidor, de aplicación, de actividad de usuario y de monitoreo.
- Exporte los logs con **marca de tiempo y nivel**; son los campos que más valor aportan al análisis.
- Conserve los originales en modo solo lectura y trabaje sobre copias. Nombre los archivos con tipo y fecha.
- Elija un periodo representativo y no mezcle periodos sin relación. Anonimice datos personales si el informe se compartirá.

Archivo	Tipo	Periodo	Entradas	Err.	Adv.
Server_Log_Input_1	Servidor	2024-04-25 09:00 a 11:50	35	12	8
Server_Log_Input_2	Servidor	2024-06-01 00:01 a 09:45	37	6	11
User_Activity_Log_Input_3	Actividad de usuario	2024-06-01 09:00 a 09:45	21	0	2
Application_Debugging_Log_Input_4	Depuración de aplicación	2024-06-01 10:01 a 10:08	26	3	3
Application_Debugging_Log_Input_5	Depuración de aplicación	2024-06-23 10:00 a 10:27	32	6	3
System_Monitoring_Log_Input_6	Monitoreo de sistema	2024-06-24 07:00 a 09:00	29	5	6
Total			180	32	33

Atención: estos archivos cubren periodos separados (abril y junio de 2024). Al combinarlos, la herramienta omite las tendencias entre archivos porque serían engañosas, y recomienda --por-archivo.

Paso 3. Procesamiento previo de datos

logsum aplica automáticamente las siguientes reglas antes de contar. Revíselas para saber cómo se interpretan sus logs:

Regla	Qué hace	Ejemplo
Extracción de fecha	Admite formatos ISO, Apache, nginx y syslog; ignora fracciones de segundo y zona horaria.	2024-06-01 10:04:00,123 se lee como 2024-06-01 10:04
Unificación de niveles	WARN y WARNING, ERR y ERROR; FATAL y SEVERE pasan a CRITICAL.	WARN pasa a WARNING
Trazas multilínea	Las líneas sin fecha se adjuntan a la entrada previa; se conserva la línea de la excepción.	NullPointerException en CartService
Normalización	Reemplaza datos variables por marcadores: <IP>, <PATH>, <N>, <UUID>, <HEX>, <EMAIL>.	Memory usage at 80% pasa a Memory usage at <N>%
Nivel inferido	Si hay fecha pero no nivel, usa palabras clave (error, failed, denied, timeout...).	"Failed password" se clasifica como WARNING
Líneas no reconocidas	Se omiten y se reflejan en la diferencia entre líneas leídas y entradas.	Línea suelta "20" al final del archivo 5

Verificaciones manuales previas: codificación UTF-8, archivos vacíos, zona horaria consistente entre fuentes [confirmar] y que los logs no contengan datos sensibles sin proteger.

Paso 4. Entrenamiento y refinamiento de la herramienta

logsum **no utiliza aprendizaje automático**, así que no hay un modelo que entrenar. El “entrenamiento” consiste en **calibrar las reglas con sus propios logs** mediante ciclos cortos: **(1)** ejecutar sobre una muestra, **(2)** revisar si los patrones están demasiado fragmentados o demasiado agrupados, **(3)** ajustar reglas o parámetros, **(4)** volver a ejecutar y comparar y **(5)** documentar el cambio.

Síntoma	Ajuste recomendado
Demasiados patrones casi idénticos (en el ejemplo: 56 patrones para 65 problemas)	Añadir reglas en la lista NORMALIZERS (por ejemplo, para nombres entre comillas) o apoyarse en el resumen por componente.
Mensajes de distinta gravedad quedan en un mismo patrón	No normalizar el dato que distingue la gravedad (los códigos HTTP se conservan como HTTP_404, HTTP_500).
Fechas o milisegundos aparecen dentro del mensaje	Ampliar la expresión regular de fecha en TS_PATTERNS.
Demasiado ruido de niveles bajos	Subir el nivel mínimo con --min-level ERROR o limitar con --top.
Falsos picos o picos que no se detectan	Ajustar el umbral de picos (media + 2.5 desviaciones, mínimo 5 eventos) en build_report.
Uso de memoria alto con logs muy variados	Ajustar la constante MAX_PATTERNS.

Registro de refinamientos aplicados en este proyecto

Hallazgo durante las pruebas	Corrección
Los milisegundos del timestamp (“,123”) se colaban en el mensaje.	Expresión de fecha ampliada.
Los códigos HTTP 404 y 500 se agrupaban en un solo patrón.	El código se conserva en el patrón (HTTP_404).
Se perdía el corchete inicial y quedaba “db]”; “HTTP_404” se normalizaba a medias.	Limpieza de mensaje y regla numérica corregidas.
Los 6 archivos reales cubrían periodos separados y las tendencias eran engañosas.	Detección de periodos separados, resumen por archivo, resumen por componente y opción --por-archivo.

Paso 5. Pruebas y evaluación

Valide la herramienta antes de confiar en sus conclusiones. Resultados del plan de pruebas de referencia:

Prueba	Datos y método	Resultado	Estado
T1. Funcional multi-formato	6,566 líneas sintéticas: texto, JSON .gz y acceso web, con patrones inyectados.	Detectó los patrones y la tendencia CRECIENTE; reveló 2 defectos, ya corregidos.	Cumple
T2. Logs reales	6 archivos (184 líneas). Conteo independiente con grep por archivo.	180 entradas reconocidas (97.8%). Errores y advertencias coinciden en los 6 archivos (32 y 33). Las 4 líneas restantes: 3 de traza y 1 suelta.	Cumple
T3. Volumen	1,000,000 de líneas (55 MB) generadas.	23 s en el entorno de pruebas; 130,355 problemas agrupados en 14 patrones.	Cumple (meta sugerida: [definir])
T4. Modo --ai	Requiere ANTHROPIC_API_KEY.	No ejecutada: solo se verificó el aviso cuando falta el SDK.	Pendiente
T5. Revisión de un analista	Un analista valida patrones y tendencias con sus logs.	[A completar]	Pendiente

Métricas sugeridas: cobertura de parseo (entradas / líneas), exactitud de conteos frente a un conteo independiente, tiempo por millón de líneas, proporción de tendencias confirmadas por un analista y tiempo de diagnóstico antes y después de usar la herramienta.

4. Principales desafíos y soluciones

Desafío	Impacto	Solución
Formatos heterogéneos de log	Líneas ignoradas o mal clasificadas	Varios parsers, nivel inferido por palabras clave y conteo de líneas no reconocidas.
Mensajes con datos variables	Un mismo problema aparece como decenas de mensajes	Normalización de IP, rutas y números; resumen por componente.
Trazas de pila multilínea	Entradas falsas y conteos inflados	Las líneas sin fecha se adjuntan a la entrada previa.
Grandes volúmenes	Consumo de memoria y tiempo	Streaming, límite de patrones y soporte .gz (1 millón de líneas en unos 23 s).
Periodos separados en el tiempo	Tendencias engañosas entre archivos	Detección automática y análisis independiente con --por-archivo.
Eventos de negocio registrados como fallos (tarjeta inválida, fondos insuficientes)	Se sobreestiman las fallas del sistema	Registrarlos como INFO o con un componente propio; filtrarlos al interpretar.
Niveles inconsistentes y trazas en DEBUG	Se pierde información clave en producción	Unificar niveles y registrar las trazas al mismo nivel que el error.
Datos sensibles (IP, correos, usuarios)	Riesgo de privacidad	Análisis local. Con --ai solo se envía el resumen, pero incluye un ejemplo por patrón: revíselo o anonimícelo antes.
Líneas truncadas o corruptas	Ruido en los datos	Se omiten y se cuentan; revisar la fuente del log si son frecuentes.

5. Casos de uso potenciales

Caso de uso	Descripción	Salida clave
Triaje de incidentes	Priorizar en minutos qué falla más y desde cuándo durante una caída.	Errores, picos y línea de tiempo
Revisión posterior al incidente	Reconstruir qué componentes fallaron y en qué orden.	Componentes y patrones
Detección de problemas sistémicos	Encontrar fallos que reaparecen entre días o archivos (ej. payment-service el 25 abr y el 1 jun).	Componentes con más problemas
Seguridad	Detectar intentos de acceso anómalos (ej. 8 inicios de sesión fallidos desde 203.0.113.45; acceso bloqueado desde 198.51.100.23).	Advertencias por patrón
Planificación de capacidad	Seguir la saturación de memoria, CPU y disco (ej. /dev/sda1: 15%, 12% y luego 10% libre).	Patrones CRECIENTE o CONSTANTE
Auditoría de calidad del logging	Detectar ruido, niveles inconsistentes y brechas en los registros.	Hallazgos (Anexo A)
Informes periódicos y validación de despliegues	Ejecutar la herramienta de forma programada o tras cada despliegue y comparar contra la línea base.	Informe .md y JSON

6. Instrucciones para el usuario

6.1 Instalación

- Copie `logsum.py` a una carpeta de trabajo. No requiere instalación ni librerías externas.
- Verifique la versión: `python --version` (3.8 o superior).
- Opcional, para `--ai`: `pip install anthropic` y defina la variable de entorno `ANTHROPIC_API_KEY`.

6.2 Comandos básicos

```
# Un archivo
python logsum.py Server_Log_Input_1.txt

# Varios archivos, todos los patrones, guardando el informe
python logsum.py *.txt --top 0 --out reporte.md

# Solo errores, con salida JSON para otras herramientas
python logsum.py /var/log/miapp/ --min-level ERROR --json reporte.json

# Un informe independiente por archivo
python logsum.py *.txt --por-archivo

# Desde la entrada estándar y con interpretación de Claude
cat app.log | python logsum.py -
python logsum.py app.log --ai
```

6.3 Referencia de opciones

Opción	Descripción	Por defecto
<code> rutas </code>	Archivos, carpetas o “-” (entrada estándar). Admite .gz.	Obligatorio
<code> --top N </code>	Patrones mostrados por sección; 0 muestra todos.	10
<code> --min-level </code>	Nivel mínimo considerado problema: DEBUG, INFO, WARNING, ERROR o CRITICAL.	WARNING
<code> --out ARCHIVO </code>	Guarda el informe en Markdown.	Solo consola
<code> --json ARCHIVO </code>	Guarda los datos estructurados.	Desactivado
<code> --por-archivo </code>	Genera un informe independiente por cada archivo.	Desactivado
<code> --ai </code>	Añade la interpretación de Claude (solo en modo combinado).	Desactivado

6.4 Cómo leer el informe

Sección	Qué responde	Acción sugerida
Distribución por nivel	¿Qué proporción de las entradas son problemas?	Compare contra su línea base.
Resumen por archivo	¿Qué fuente concentra más problemas?	Empiece por la fuente más afectada.
Errores y Advertencias	¿Cuáles son los mensajes únicos y cuánto se repiten?	Ataque primero los más frecuentes.
Componentes con más problemas	¿Qué módulos reinciden?	Asigne un responsable por componente.
Patrones y tendencias	¿Algo empeora, es nuevo o es constante?	Investigue CRECIENTE y CONSTANTE.
Picos y línea de tiempo	¿Cuándo ocurrió y qué lo provocó?	Correlacione con despliegues y cambios.

6.5 Solución de problemas

Mensaje o síntoma	Causa probable y solución
"No se encontraron entradas de log reconocibles"	Las líneas no tienen fecha ni nivel. Revise el formato o añada una expresión de fecha en TS_PATTERNS.
"Tendencias omitidas: periodos separados"	Los archivos están muy distanciados en el tiempo. Use --por-archivo.
Muchos patrones casi iguales	Ver Paso 4: añada reglas de normalización o use el resumen por componente.
--ai muestra un aviso	Instale el SDK (pip install anthropic) y defina ANTHROPIC_API_KEY.

7. Escenarios de ejemplo

Escenario 1. Triage de un servidor inestable

Contexto: el 25 de abril de 2024 el servidor presentó caídas y reinicios entre las 09:00 y las 11:50. Se necesita saber en minutos qué falla más.

```
python logsum.py Server_Log_Input_1.txt --top 0 --out triage_25abr.md
```

Resultado: 35 entradas, 12 errores y 8 advertencias (20 problemas en 11 patrones).

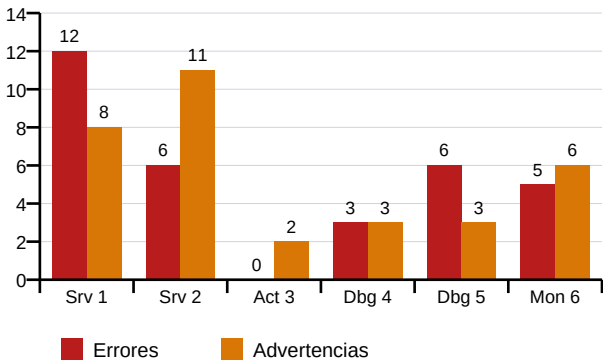
Hallazgo	Evidencia	Hipótesis o acción (a validar)
La base de datos concentra la mitad de los errores	DatabaseConnect: 4 (3 timeouts, aprox. uno por hora, y 1 de autenticación) más NetworkDisconnect: 2 (db-01). Son 6 de 12.	Revisar conectividad y credenciales hacia db-01; CPU de db-03 al 90% y 92%.
Servicios que caen	ServiceCrash: email-service a las 09:50 y 11:30; payment-service a las 10:45.	Analizar la causa raíz y añadir supervisión con reinicio.
Configuración no corregida	FileNotFound: app.config ausente a las 09:20 y otra vez a las 11:15.	Restaurar el archivo en /etc/app/.
Recursos al límite	Disco /dev/sda1: 15% a 12% libre; memoria de web-02: 80% a 85%; latencia de eth1: 300 y 280 ms.	Liberar espacio y revisar el consumo de memoria.
Ciclos de reinicio	3 arranques (09:00, 10:00, 10:55) y 3 apagados por mantenimiento o actualizaciones.	Confirmar si fueron planificados.

Escenario 2. Revisión consolidada para detectar problemas sistémicos

Contexto: el equipo quiere saber si los seis archivos de log comparten causas de fondo. Se ejecuta el análisis combinado y, para tendencias, el análisis por archivo.

```
python logsum.py *.txt --top 0 --out reporte_NetGuardPro.md
python logsum.py *.txt --por-archivo
```

Problemas por archivo de log



Componentes con más problemas

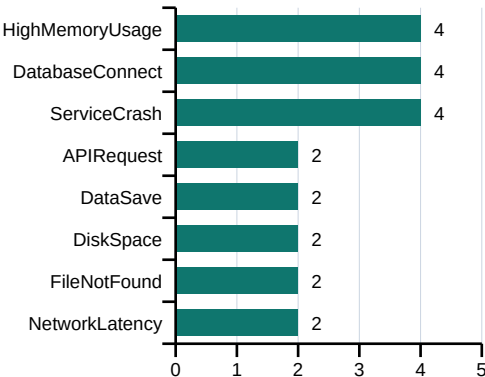


Figura 1. Problemas por archivo (izquierda) y componentes con más problemas (derecha). Total: 65 problemas en 56 patrones distintos.

Tema sistémico	Evidencia	Acción sugerida
Base de datos	Unos 10 de 32 errores (31%): timeouts y desconexiones, "Database write error", "Lock wait timeout", product-db sin respuesta; consulta de 450 ms.	Revisar capacidad, bloqueos y red hacia la base de datos.
Red	Latencias de 300, 280, 500 y 350 ms; 5% de errores de red; timeouts en backup y consulta de datos.	Revisar enlaces y tiempos de espera.
Saturación de recursos	6 advertencias de memoria; fuga en backup_service; data_aggregator terminado por exceder la memoria; /dev/sda1 en 15%, 12% y 10%.	Fijar límites, alertas y limpieza programada.
Servicios que caen	payment-service (25 abr y 1 jun), email-service, user-auth-service y report-generator.	Supervisión de salud y análisis de causa raíz.
Errores sin recuperación registrada	Guardado del pedido 56789 falla tras el reintento; NullPointerException en CartService (línea 85).	Registrar el resultado final de cada operación.
Calidad del logging	Eventos de negocio como ERROR, traza en DEBUG, WARN y WARNING mezclados y una línea truncada.	Ajustar la configuración del logger.

Siguiente paso: priorizar la base de datos y la red, y compartir el informe con los responsables de cada componente. Opcionalmente, ejecute - -a.i para obtener hipótesis de causa raíz sobre el resumen.

Anexo A. Plantilla de registro de hallazgos

ID	Patrón o componente	Frecuencia	Severidad	Acción propuesta	Responsable / Estado
H-01	DatabaseConnect	4 (3 timeouts, 1 autenticación)	Alta	Verificar conectividad y credenciales hacia db-01	[Nombre] / Abierto
H-02	[]	[]	[]	[]	[]
H-03	[]	[]	[]	[]	[]
H-04	[]	[]	[]	[]	[]

Anexo B. Lista de verificación antes de entregar un análisis

[]	Los objetivos del análisis están definidos y documentados.
[]	Los logs originales se conservaron sin cambios y no contienen datos sensibles sin proteger.
[]	Se revisó la diferencia entre líneas leídas y entradas reconocidas.
[]	Se confirmaron al menos los 3 patrones principales contra los logs originales.
[]	Las tendencias se interpretaron solo dentro de periodos comparables.
[]	Las acciones tienen responsable y fecha.

Anexo C. Glosario

Término	Definición
Patrón	Grupo de mensajes iguales salvo por sus datos variables (IP, números, rutas).
Componente	Etiqueta del módulo que emite el mensaje (ej. DatabaseConnect, ServiceCrash).
Ráfaga	Patrón concentrado en un solo intervalo de tiempo.
Tendencia	Evolución de un patrón: creciente, decreciente, constante o nuevo.
Pico	Intervalo cuyo número de problemas supera ampliamente el promedio.

Anexo D. Control de cambios

Versión	Fecha	Autor	Descripción
1.0	2026-10-05	[Nombre]	Versión inicial de la guía con el análisis de referencia de seis archivos de log.
[]	[]	[]	[]